Universidad Nacional Autónoma de México

Facultad de Ingeniería

Laboratorio de Arquitectura y Organización de Computadoras

Grupo: 07 - Semestre: 2027-1

Reporte Práctica 6 pt 2.

Secuenciador básico de 4 microinstrucciones

Fecha de Entrega

16 de Octubre del 2026

Profesor

Ing. Julio Cesar Cruz Estrada

Integrantes:

Espinoza Matamoros Percival Ulises

Montiel Juárez Oscar Iván

Veraza García Amy Valentina

1. Objetivo

Implementar un secuenciador básico microprogramado a partir de una carta ASM, utilizando una memoria ROM para almacenar las microinstrucciones, un contador de microprograma y la lógica necesaria para seleccionar la siguiente dirección.

2. Desarrollo

En esta práctica se construyó un secuenciador microprogramado que ejecuta la carta ASM proporcionada. A diferencia de la versión anterior, la secuencia no se determina con direcciones fijas: cada estado consulta una palabra de la ROM y, a partir de sus campos, obtiene las salidas del estado, la condición que se debe probar y la fuente de la siguiente microdirección.

El contador de microprograma (`mPC`) guarda la dirección presente. Esa dirección consulta la ROM, se separa en campos y alimenta la ruta de control. El bloque de prueba selecciona una de las entradas X, Y, `INT` o una constante, mientras que el valor VF permite invertir la condición por medio de una compuerta XOR. Finalmente, el campo MI decide si la siguiente dirección corresponde al paso continuo, la liga, el registro de transformación o el registro de interrupción.

2.1. Carta ASM y tamaño de la memoria

La Figura 1 representa la carta ASM implementada y conserva la distribución de la carta proporcionada: la rama izquierda corresponde a la prueba X, la rama central a la prueba Y y la rama derecha a la interrupción. Los rombos marcados con T representan los saltos de transformación. Desde E_1, $T = 00, 11$ llega a E_2, $T = 01$ llega a E_8 y $T = 10$ llega a E_{13}; desde E_{11}, $T = 01$ llega a E_7 y los demás valores llegan a E_{12}. La posición E_{15} se usa solamente como retorno de seguridad de la ROM, por lo que no forma parte de la carta ASM original.

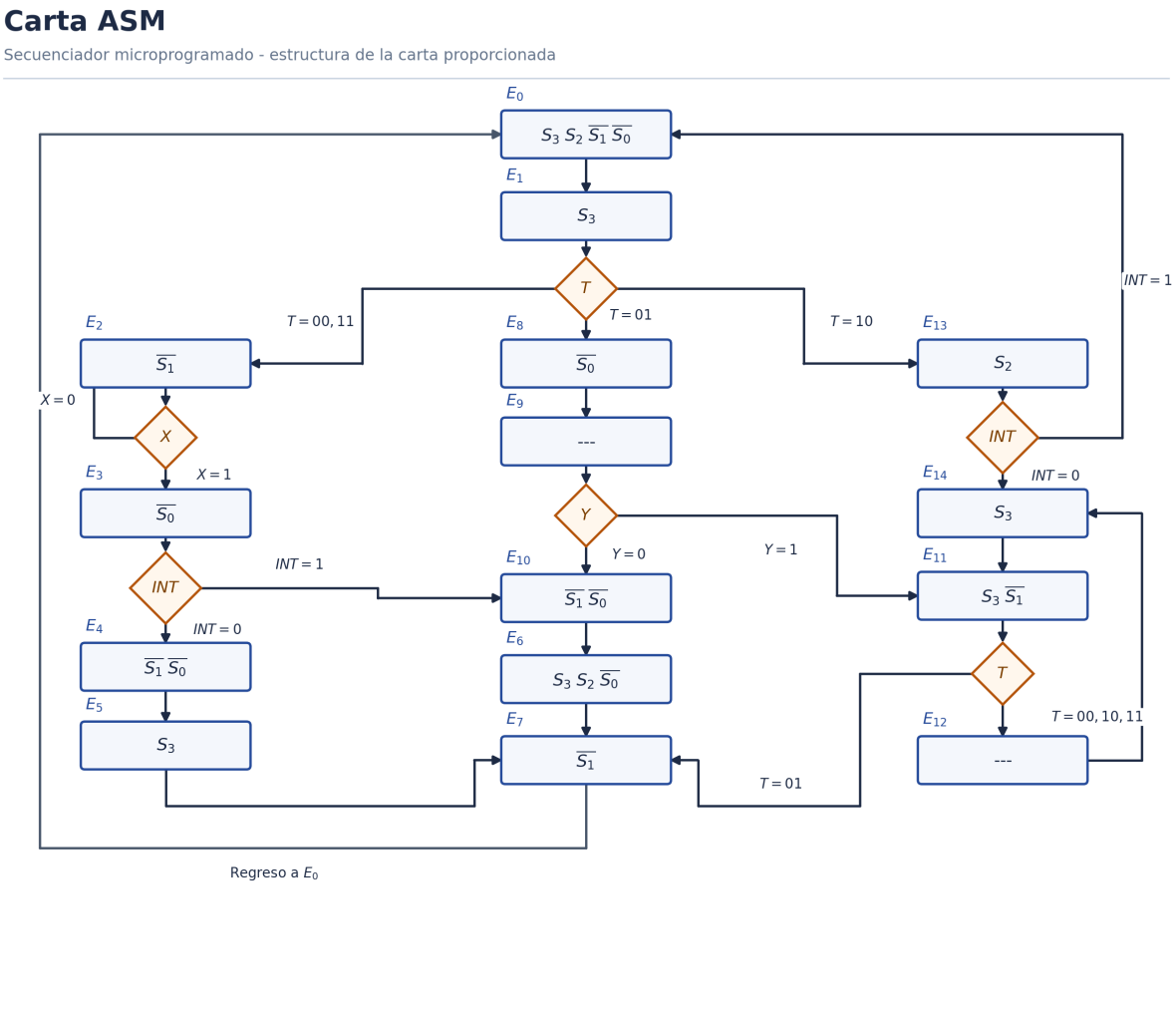


Figura 1: Carta ASM del secuenciador microprogramado, redibujada a partir de la carta proporcionada.

La memoria requiere cuatro bits de dirección, ya que la microprogramación ocupa las posiciones de 0 a 15. Cada microinstrucción tiene un campo de prueba de dos bits, un bit de valor de prueba, dos bits para el tipo de microinstrucción, cuatro bits de liga y cuatro bits de salidas. Por lo tanto, su organización es de 16 palabras por 13 bits:

$$n = 2^4 = 16 \text{ palabras}, \qquad m = 2 + 1 + 2 + 4 + 4 = 13 \text{ bits}$$

$$\text{Capacidad} = 16 \times 13 = \boxed{208 \text{ bits}}.$$

Tabla 1: Formato de una microinstrucción de la memoria de control.

Campo	Prueba	VF	MI	Liga	Salidas
Bits	K_1K_0	1	I_1I_0	$L_3L_2L_1L_0$	$S_3S_2S_1S_0$
Longitud	2	1	2	4	4

Los valores de MI codifican cuatro opciones: `00` selecciona $mPC + 1$, `01` selecciona la liga cuando la condición se cumple, `10` selecciona el registro de transformación y `11` selecciona el registro de interrupción. El bloque `muxEntradas` define la entrada que se prueba: `00` corresponde a X, `01` a Y, `10` a `INT` y `11` a una constante cero.

2.2. Contenido de la memoria de control

La Tabla 2 contiene la microprogramación capturada en el módulo `rom.vhd`. Los campos indiferentes de la carta ASM se guardaron con un valor definido, por lo que cada dirección de la ROM entrega una palabra completa de 13 bits.

Tabla 2: Contenido de la ROM para la carta ASM.

Dirección	Prueba	VF	MI	Liga	Salidas	Operación / siguiente estado
`0000`	`11`	0	`00`	`0000`	`1100`	Paso continuo a E_1.
`0001`	`11`	0	`10`	`0000`	`1011`	Transformación desde E_1.
`0010`	`00`	0	`01`	`0010`	`0001`	Prueba X: $0 \to E_2$, $1 \to E_3$.
`0011`	`10`	1	`11`	`0000`	`0000`	Salto por interrupción desde E_3.
`0100`	`11`	0	`00`	`0000`	`0000`	Paso continuo a E_5.
`0101`	`11`	0	`01`	`0000`	`1011`	Retorno incondicional a E_0.
`0110`	`11`	0	`00`	`0000`	`1110`	Paso continuo a E_7.
`0111`	`11`	0	`01`	`0000`	`0001`	Retorno incondicional a E_0.
`1000`	`11`	0	`00`	`0000`	`0010`	Paso continuo a E_9.
`1001`	`01`	1	`01`	`1011`	`0011`	Prueba Y: $1 \to E_{11}$, $0 \to E_{10}$.
`1010`	`11`	0	`01`	`0110`	`0000`	Salto incondicional a E_6.
`1011`	`11`	0	`10`	`0000`	`1001`	Transformación desde E_{11}.
`1100`	`11`	0	`01`	`1110`	`0011`	Salto incondicional a E_{14}.
`1101`	`10`	1	`11`	`0000`	`0111`	Salto por interrupción desde E_{13}.
`1110`	`11`	0	`01`	`1011`	`1011`	Salto incondicional a E_{11}.
`1111`	`11`	0	`01`	`0000`	`0011`	Posición de seguridad: regreso a E_0.

2.3. Arquitectura implementada

El bloque principal `practica6` integra el generador de paso por botón, el registro `mPC`, el incrementador, la ROM, el divisor de campos, los multiplexores y los registros de transfor-

mación e interrupción. El contador únicamente cambia cuando se genera el pulso `paso`; de esta manera cada pulsación permite observar una microinstrucción en la tarjeta.

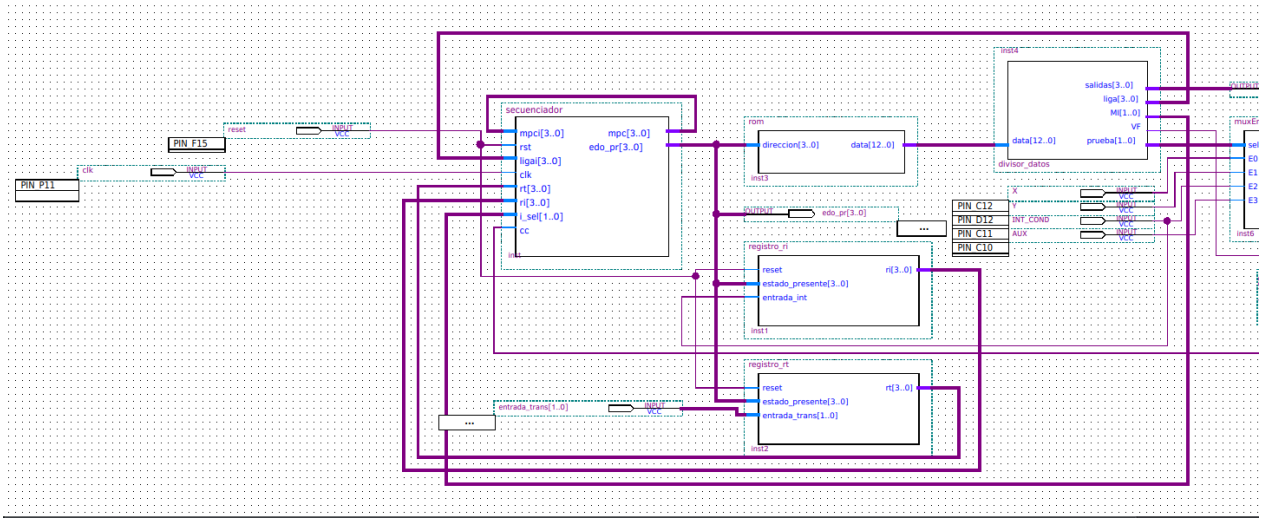


Figura 2: Interconexión de la ruta de microdirección del secuenciador.

La Figura 3 muestra la conexión de la ROM con el divisor de datos. La palabra de 13 bits se separa en los campos de prueba, VF, MI, liga y salidas. Las salidas llegan directamente a los LEDs, mientras que los demás campos controlan la selección de la siguiente microdirección.

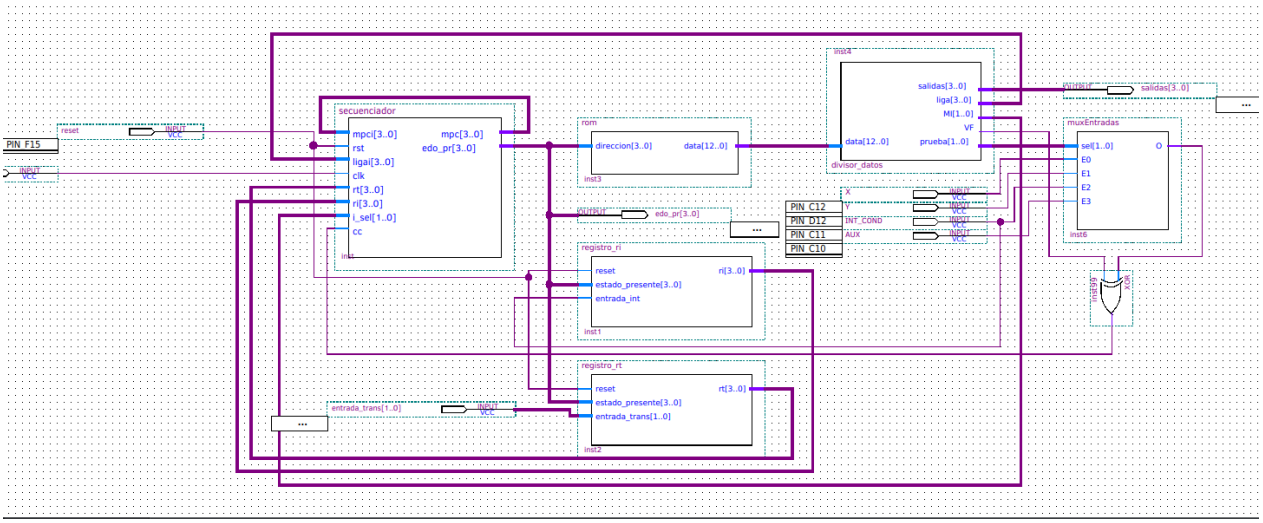


Figura 3: Interconexión de la memoria ROM, el divisor de datos y la ruta de control.

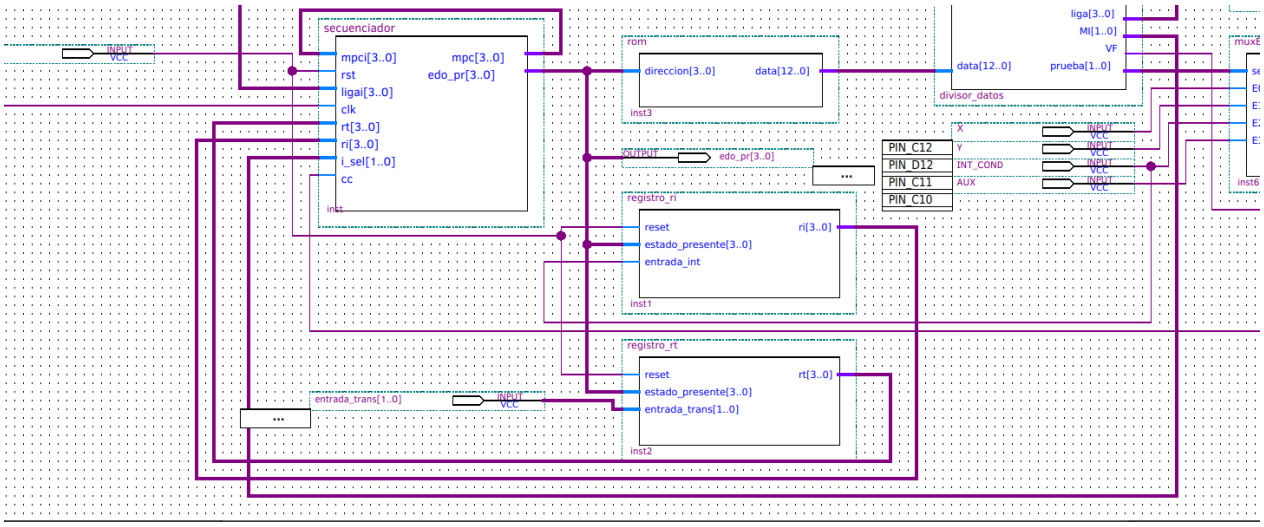


Figura 4: Diagrama final de bloques del secuenciador microprogramado.

2.4. Módulos implementados en VHDL

Módulo principal. El módulo `practica6` conecta la entrada de reloj, las entradas de prueba X, Y e `INT`, la entrada de transformación y los LEDs. La señal `cc` se obtiene con una XOR entre la condición seleccionada y VF.

Listing 1: Conexión estructural del secuenciador microprogramado.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Conexion unica del secuenciador de la carta ASM.
entity practica6 is
    generic (CUENTA_REBOTE : natural := 999999);
    Port (
        clk            : in  std_logic; -- MAX10_CLK1_50
        boton          : in  std_logic; -- KEY0: avanzar, activo en 0
        rst            : in  std_logic; -- KEY1: regresar a estado 0, activo en 0
        X              : in  std_logic; -- SW2
        Y              : in  std_logic; -- SW1
        entrada_int_RI : in  std_logic; -- SW3
        entrada_trans  : in  std_logic_vector(1 downto 0); -- SW5:SW4
        edo_pr         : out std_logic_vector(3 downto 0); -- LEDR9:6
        salidas        : out std_logic_vector(3 downto 0)  -- LEDR3:0
    );
end practica6;

architecture Estructural of practica6 is
    signal paso, vf, condicion, cc : std_logic;
    signal mpc, mpc_mas_uno, liga, rt, ri, siguiente : std_logic_vector(3 downto 0);
    signal dato_rom : std_logic_vector(12 downto 0);
    signal prueba, mi, sel_direccion : std_logic_vector(1 downto 0);
begin
    tecla : entity work.paso_boton
        generic map (CUENTA_REBOTE => CUENTA_REBOTE)
        port map (clk => clk, reset => rst, boton => boton, paso => paso);

```

```
    contador : entity work.registro_mpc
        port map (reloj => clk, reset => rst, habilitar => paso,
                  entrada_r => siguiente, mpc => mpc);

    incrementa : entity work.incrementador
        port map (edop => mpc, edouno => mpc_mas_uno);

    memoria : entity work.rom
        port map (direccion => mpc, data => dato_rom);

    campos : entity work.divisor_datos
        port map (data => dato_rom, prueba => prueba, VF => vf,
                  MI => mi, liga => liga, salidas => salidas);

    -- AUX es cero fijo en la carta; asi los saltos por Liga son incondicionales.
    entradas : entity work.muxEntradas
        port map (sel => prueba, E0 => X, E1 => Y,
                  E2 => entrada_int_RI, E3 => '0', O => condicion);
    cc <= condicion xor vf;

    control : entity work.logica
        port map (i_sel => mi, cc => cc, sel => sel_direccion);

    transformacion : entity work.registro_rt
        port map (reset => rst, estado_presente => mpc,
                  entrada_trans => entrada_trans, rt => rt);

    interrupcion : entity work.registro_ri
        port map (reset => rst, estado_presente => mpc,
                  entrada_int => entrada_int_RI, ri => ri);

    direccion : entity work.muxDireccion
        port map (sel => sel_direccion, E0 => mpc_mas_uno, E1 => liga,
                  E2 => rt, E3 => ri, sal => siguiente);

    edo_pr <= mpc;
end Estructural;
```

Memoria y divisor de campos. La ROM contiene las dieciséis microinstrucciones de la Tabla 2. El divisor entrega por separado los campos que usa cada bloque de la arquitectura.

Listing 2: Memoria ROM de 16 palabras por 13 bits.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity rom is
    Port (
        direccion : in  std_logic_vector(3 downto 0);
        data      : out std_logic_vector(12 downto 0)
    );
end rom;

architecture Behavioral of rom is

    type mem is array (0 to 15) of std_logic_vector(12 downto 0);

    -- Formato de la microinstruccion (13 bits):
    -- data(12 downto 11) = Prueba K1,K0
```

```
    -- data(10)           = VF
    -- data(9 downto 8)   = MI M1,M0
    -- data(7 downto 4)   = Liga L3,L2,L1,L0
    -- data(3 downto 0)   = Salidas S3,S2,S1,S0
    --
    -- MI: 00 = avance (MPC+1), 01 = salto condicional por Liga,
    --     10 = salto a RT,     11 = salto por interrupcion a RI.
    -- Los '*' de la tabla del reporte se sustituyen por ceros o por
    -- una prueba sin uso, ya que son campos indiferentes en esa fila.
    constant ROM : mem := (
        -- Dir  Prueba VF MI  Liga  Salidas
        0  => "11" & "0" & "00" & "0000" & "1100", -- Carta ASM: S3 S2 /S1 /S0
        1  => "11" & "0" & "10" & "0000" & "1011",
        2  => "00" & "0" & "01" & "0010" & "0001",
        3  => "10" & "1" & "11" & "0000" & "0000",
        4  => "11" & "0" & "00" & "0000" & "0000",
        5  => "11" & "0" & "01" & "0000" & "1011",
        6  => "11" & "0" & "00" & "0000" & "1110",
        7  => "11" & "0" & "01" & "0000" & "0001",
        8  => "11" & "0" & "00" & "0000" & "0010",
        9  => "01" & "1" & "01" & "1011" & "0011",
        10 => "11" & "0" & "01" & "0110" & "0000",
        11 => "11" & "0" & "10" & "0000" & "1001",
        12 => "11" & "0" & "01" & "1110" & "0011",
        13 => "10" & "1" & "11" & "0000" & "0111",
        14 => "11" & "0" & "01" & "1011" & "1011",
        15 => "11" & "0" & "01" & "0000" & "0011"
    );

begin

    process(direccion)
    begin
        data <= ROM(conv_integer(unsigned(direccion)));
    end process;

end Behavioral;
```

Listing 3: Divisor de los campos de una microinstrucción.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity divisor_datos is
    Port (
        data    : in  std_logic_vector(12 downto 0);
        prueba  : out std_logic_vector(1 downto 0);
        VF      : out std_logic;
        MI      : out std_logic_vector(1 downto 0);
        liga    : out std_logic_vector(3 downto 0);
        salidas : out std_logic_vector(3 downto 0)
    );
end divisor_datos;

architecture Behavioral of divisor_datos is
begin
    -- Prueba[12:11], VF[10], MI[9:8], Liga[7:4], Salidas[3:0]
    prueba  <= data(12 downto 11);
    VF      <= data(10);
    MI      <= data(9 downto 8);
    liga    <= data(7 downto 4);
    salidas <= data(3 downto 0);
end Behavioral;
```

Selección de condiciones y direcciones. `muxEntradas` selecciona la condición indicada por el campo `Prueba`; `logica` interpreta el campo MI y `muxDireccion` selecciona la fuente de la siguiente dirección.

Listing 4: Multiplexor de entradas para la condición de control.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity muxEntradas is
    Port (
        sel : in  std_logic_vector(1 downto 0);
        E0  : in  std_logic; -- X
        E1  : in  std_logic; -- Y
        E2  : in  std_logic; -- INT
        E3  : in  std_logic; -- AUX
        O   : out std_logic
    );
end muxEntradas;

architecture Behavioral of muxEntradas is
begin
    process(sel, E0, E1, E2, E3)
    begin
        O <= '0';
        case sel is
            when "00"   => O <= E0;
            when "01"   => O <= E1;
            when "10"   => O <= E2;
            when "11"   => O <= E3;
            when others => O <= '0';
        end case;
    end process;
end Behavioral;
```

Listing 5: Lógica que decodifica el tipo de microinstrucción.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity logica is
    Port (
        i_sel : in  std_logic_vector(1 downto 0); -- i_sel(1)=i_1, i_sel(0)=i_0
        cc    : in  std_logic;                    -- Condición de control/prueba
        sel   : out std_logic_vector(1 downto 0)  -- Selector de modo
    );
end logica;

architecture Behavioral of logica is
begin
    process(i_sel, cc)
    begin
        sel <= "00";
        case i_sel is
            when "00" =>
                sel <= "00";
            when "01" =>
                if cc = '0' then
                    sel <= "01"; -- Salto condicional a Liga de la ROM
                else
                    sel <= "00"; -- Paso continuo
                end if;
```

```
            when "10" =>
                sel <= "10"; -- Salto a transformacion
            when "11" =>
                sel <= "11"; -- RI decide la direccion con estado e INT
            when others =>
                sel <= "00";
        end case;
    end process;
end Behavioral;
```

Listing 6: Multiplexor de la siguiente microdirección.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity muxDireccion is
    Port (
        sel : in  std_logic_vector(1 downto 0);
        E0  : in  std_logic_vector(3 downto 0); -- 00: MPC + 1
        E1  : in  std_logic_vector(3 downto 0); -- 01: Liga de la ROM
        E2  : in  std_logic_vector(3 downto 0); -- 10: Transformacion
        E3  : in  std_logic_vector(3 downto 0); -- 11: Interrupcion
        sal : out std_logic_vector(3 downto 0)  -- siguiente direccion del MPC
    );
end muxDireccion;

architecture Behavioral of muxDireccion is
begin
    process(sel, E0, E1, E2, E3)
    begin
        sal <= E0;
        case sel is
            when "00"   => sal <= E0;
            when "01"   => sal <= E1;
            when "10"   => sal <= E2;
            when "11"   => sal <= E3;
            when others => sal <= E0;
        end case;
    end process;
end Behavioral;
```

Registros y paso por botón. El registro `mPC` se actualiza en el flanco ascendente del reloj cuando recibe la habilitación del bloque `paso_boton`. Los módulos `registro_rt` y `registro_ri` generan los destinos que corresponden a los rombos de transformación e interrupción de la carta ASM.

Listing 7: Registro del contador de microprograma.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity registro_mpc is
    Port (
        reloj     : in  std_logic;
        reset     : in  std_logic; -- KEY1, activo en 0
        habilitar : in  std_logic; -- Un pulso por pulsacion de KEY0
        entrada_r : in  std_logic_vector(3 downto 0);
        mpc       : out std_logic_vector(3 downto 0)
    );
```

```
end registro_mpc;

architecture Behavioral of registro_mpc is
    signal estado : std_logic_vector(3 downto 0) := "0000";
begin
    process(reloj, reset)
    begin
        if reset = '0' then
            estado <= "0000";
        elsif rising_edge(reloj) then
            if habilitar = '1' then
                estado <= entrada_r;
            end if;
        end if;
    end process;

    mpc <= estado;
end Behavioral;
```

Listing 8: Registro de transformación.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity registro_rt is
    Port (
        reset            : in  std_logic;
        estado_presente  : in  std_logic_vector(3 downto 0);
        entrada_trans    : in  std_logic_vector(1 downto 0);
        rt               : out std_logic_vector(3 downto 0)
    );
end registro_rt;

architecture Behavioral of registro_rt is
begin
    process(reset, estado_presente, entrada_trans)
    begin
        if reset = '0' then
            rt <= "0000";
        else
            case estado_presente is
                when "0001" => -- Estado 1
                    case entrada_trans is
                        when "00"   => rt <= "0010"; -- Estado 2
                        when "01"   => rt <= "1000"; -- Estado 8
                        when "10"   => rt <= "1101"; -- Estado 13
                        when others => rt <= "0010";
                    end case;

                when "1011" => -- Estado 11
                    case entrada_trans is
                        when "00"   => rt <= "1100"; -- Estado 12
                        when "01"   => rt <= "0111"; -- Estado 7
                        when others => rt <= "1100";
                    end case;

                when others =>
                    rt <= "0000";
            end case;
        end if;
    end process;
end Behavioral;
```

Listing 9: Registro de interrupción.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity registro_ri is
    Port (
        reset            : in  std_logic;
        estado_presente  : in  std_logic_vector(3 downto 0);
        entrada_int      : in  std_logic;
        ri               : out std_logic_vector(3 downto 0)
    );
end registro_ri;

architecture Behavioral of registro_ri is
begin
    process(reset, estado_presente, entrada_int)
    begin
        if reset = '0' then
            ri <= "0000";
        else
            case estado_presente is
                when "0011" => -- Estado 3: INT=0 -> 4; INT=1 -> 10
                    if entrada_int = '0' then
                        ri <= "0100";
                    else
                        ri <= "1010";
                    end if;

                when "1101" => -- Estado 13: INT=0 -> 14; INT=1 -> 0
                    if entrada_int = '0' then
                        ri <= "1110";
                    else
                        ri <= "0000";
                    end if;

                when others =>
                    ri <= "0000";
            end case;
        end if;
    end process;
end Behavioral;
```

Listing 10: Generador de un avance por pulsación.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

-- KEY0 es activo en bajo. Un avance por pulsacion, con 20 ms de antirrebote.
entity paso_boton is
    generic (CUENTA_REBOTE : natural := 999999); -- 20 ms a 50 MHz
    Port (
        clk   : in  std_logic; -- 50 MHz
        reset : in  std_logic; -- KEY1, activo en bajo
        boton : in  std_logic; -- KEY0, activo en bajo
        paso  : out std_logic
    );
end paso_boton;

architecture Behavioral of paso_boton is
    signal meta, sinc, estable : std_logic := '1';
    signal cuenta : natural range 0 to CUENTA_REBOTE := 0;
    signal pulso : std_logic := '0';
```

```
20  begin
21      process(clk, reset)
22      begin
23          if reset = '0' then
24              meta <= '1';
25              sinc <= '1';
26              estable <= '1';
27              cuenta <= 0;
28              pulso <= '0';
29          elsif rising_edge(clk) then
30              meta <= boton;
31              sinc <= meta;
32              pulso <= '0';
33              if sinc = estable then
34                  cuenta <= 0;
35              elsif cuenta = CUENTA_REBOTE then
36                  estable <= sinc;
37                  cuenta <= 0;
38                  if sinc = '0' then
39                      pulso <= '1';
40                  end if;
41              else
42                  cuenta <= cuenta + 1;
43              end if;
44          end if;
45      end process;
46
47      paso <= pulso;
48  end Behavioral;
```

3. Simulación

La simulación verifica que el contador de microprograma actualiza `edo_pr` únicamente en el flanco donde se genera el pulso de avance. En las formas de onda se distinguen las direcciones presentes, las salidas de la ROM y las entradas que provocan los saltos de la carta ASM.

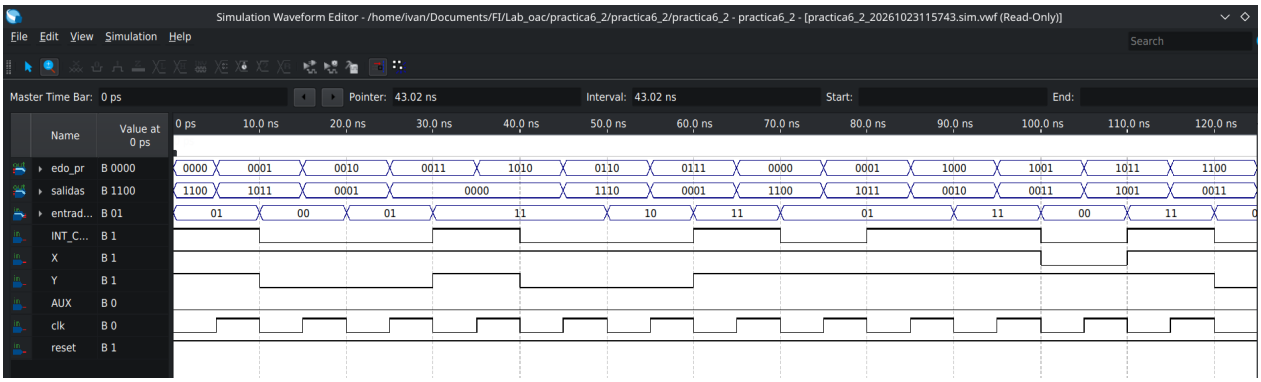


Figura 5: Simulación del recorrido de microdirecciones y de las salidas de la ROM.

En la segunda simulación se aplican diferentes valores a X, Y, `INT` y a la entrada de transformación. Con ello se comprueba que las decisiones condicionales y los registros de salto

conducen al estado definido por la carta ASM.

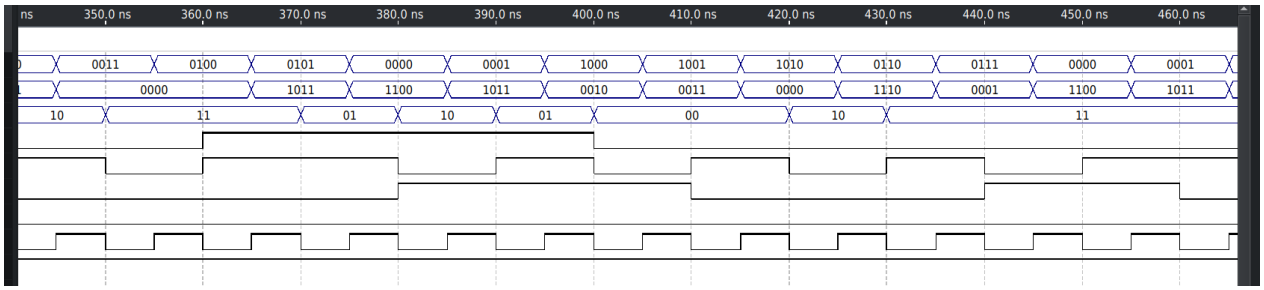


Figura 6: Verificación de condiciones, transformaciones e interrupciones.

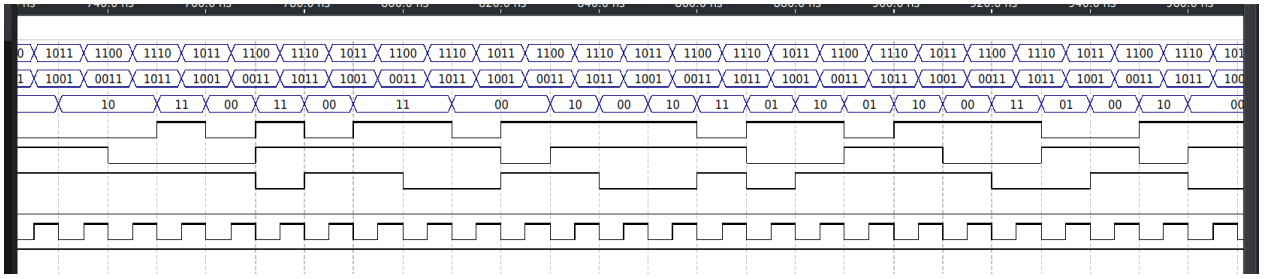
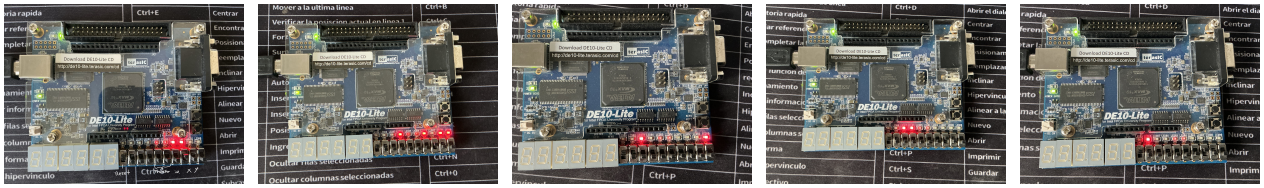


Figura 7: Continuación de la simulación con cambios de estado sincronizados.

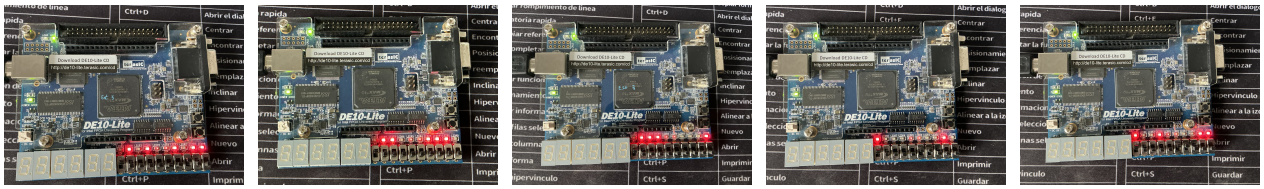
4. Resultados en la tarjeta FPGA

El circuito se descargó en la tarjeta DE10-Lite. Los LEDs de menor peso muestran el campo de salidas de la microinstrucción y los LEDs superiores permiten observar la dirección presente. Con el botón se avanza un estado por vez, por lo que es posible contrastar cada fotografía con la carta ASM y la Tabla 2.



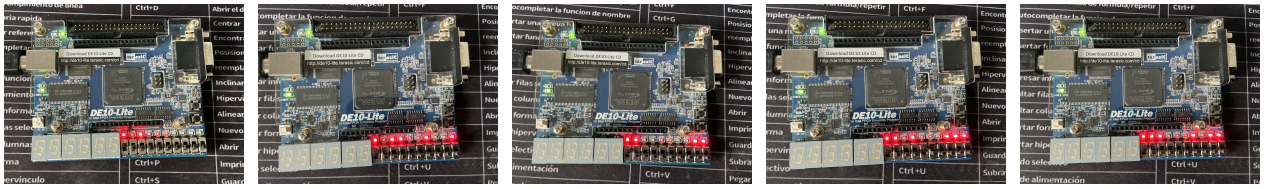
(a) E_0 (b) E_1 (c) E_2 (d) E_3 (e) E_4

Figura 8: Estados iniciales observados en la tarjeta DE10-Lite.



(a) E_5 (b) E_6 (c) E_7 (d) E_8 (e) E_9

Figura 9: Comprobación física de los estados E_5 a E_9.



(a) E_{10} (b) E_{11} (c) E_{12} (d) E_{13} (e) E_{14}

Figura 10: Comprobación física de los estados E_{10} a E_{14}.

5. Conclusión

Percival Ulises Espinoza Matamoros. Esta práctica permitió implementar una unidad de control microprogramada a partir de una carta ASM. Al almacenar las microinstrucciones en la ROM se pudo observar que una misma ruta de datos puede ejecutar diferentes recorridos solamente al cambiar el contenido de la memoria. También se comprendió la función de los campos de prueba, liga y tipo de microinstrucción.

Oscar Iván Montiel Juárez. El desarrollo permitió relacionar la carta ASM con una implementación real en VHDL. La ROM de 16×13 bits concentra la secuencia de control, mientras que el contador de microprograma y los multiplexores seleccionan la dirección siguiente. La simulación y la tarjeta mostraron que los saltos condicionados, las transformaciones y las interrupciones se realizan de acuerdo con el estado presente.

Amy Valentina Veraza García. La práctica reforzó la importancia de dividir una microinstrucción en campos independientes. Al observar los LEDs se pudo comprobar que las salidas de cada estado se obtienen directamente de la ROM y que el botón permite avanzar de forma controlada para verificar la secuencia definida en la carta ASM.